

REPUBLIQUE DU SENEGAL



Un Peuple – Un But – Une Foi

MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE

UNIVERSITE GASTON BERGER DE SAINT-LOUIS



U.F.R DE SCIENCES APPLIQUEES ET DE TECHNOLOGIE

MASTER 2 INFORMATIQUE - GDIL

Rapport de projet Text-Mining

**SUJET : Analyse et clustering plat de films à partir du dataset
TMDB**

Présenté par :

Daouda FICKOU

Mouhamadou GUEYE

Mame Anta BATHILY

Année académique: 2024/2025

Table des matières

Introduction	3
I. État de l'art	5
II. Présentation, exploration et prétraitement des données	7
A. Description détaillée du dataset TMDB (films, synopsis, genres, données numériques)	7
B. Analyse exploratoire des données	8
1. Statistiques descriptives des variables numériques	8
2. Visualisations principales	9
C. Nettoyage des données (valeurs manquantes, doublons)	11
1. Suppression des doublons	11
2. Gestion des valeurs manquantes	12
D. Prétraitement textuel des synopsis.....	12
E. Vectorisation textuelle (TF-IDF ou embeddings)	15
F. Encodage et normalisation des données numériques et catégorielles	17
III. Méthodologies	18
1- Présentation de l'Algorithme Sélectionné : Clustering K-Means.....	18
2- Justification des Choix Méthodologiques.....	19
3- Description du Paramétrage.....	20
4- Implémentation Technique : Outils et Librairies.....	21
5- Techniques de Réduction Dimensionnelle pour la Visualisation	21
IV. Analyse et Interprétation des Clusters de Films.....	22
Conclusion	27

Introduction

1. Problématique

L'essor massif des données textuelles dans le domaine du cinéma, notamment à travers les synopsis, genres, et descriptions d'acteurs des films, pose un défi important en termes d'organisation, d'analyse et d'exploitation. Face à ce volume croissant, il devient difficile d'identifier manuellement des regroupements thématiques ou des catégories utiles à la recommandation, à la recherche ou à l'analyse de tendances. Le clustering, ou classification non supervisée, offre une approche efficace pour segmenter automatiquement un large corpus de films en groupes cohérents basés sur leurs caractéristiques textuelles et numériques. La question centrale est donc : **Comment regrouper de manière pertinente les films en clusters exploitables à partir de leurs métadonnées et contenus textuels ?**

2. Objectifs et motivation

Ce projet vise à développer un modèle d'apprentissage non supervisé pour segmenter les films issus du dataset `TMDB_5000_movies` à l'aide d'algorithmes de clustering plat, en particulier **k-means**. L'objectif est d'explorer la structure sous-jacente des films, de détecter des regroupements thématiques ou commerciaux, et de voir dans quelle mesure ces clusters peuvent servir la recommandation ou l'analyse marketing. Motivé par l'intersection entre traitement automatique des langues et data science appliquée à un domaine culturel riche, ce qui permet d'allier une dimension technique et une compréhension fine de contenus audiovisuels.

3. Questions de recherche

- Comment exploiter efficacement les caractéristiques textuelles et numériques du dataset TMDB pour générer des clusters pertinents ?
- Quel est le nombre optimal de clusters qui reflète la diversité des films ?
- Dans quelle mesure les clusters issus du k-means correspondent-ils aux genres officiels ou profils d'audience ?
- Peut-on utiliser ces clusters pour améliorer des systèmes de recommandation ou la segmentation marketing ?

Quelles limites rencontrent les méthodes de clustering plat dans ce contexte et comment les dépasser ?

4. Travaux connexes

Le clustering non supervisé, notamment via l'algorithme k-means, est une technique classique en fouille de textes et data mining, largement utilisée pour structurer des corpus documentaires. Dans le contexte cinématographique, plusieurs travaux ont exploité ces méthodes pour la recommandation de films, combinant souvent la vectorisation TF-IDF des synopsis avec des métadonnées. Ces méthodes permettent d'identifier des regroupements exploitables, même si certaines limites subsistent sur la qualité et l'interprétabilité des clusters obtenus.

5. Plan du projet

- État de l'art
- Présentation du dataset TMDb, exploration et prétraitement des données.
- Description de la méthodologie de clustering utilisée, choix des algorithmes et paramétrage.
- Résultats, visualisations, évaluations quantitatives et qualitatives des clusters.
- Discussion, limite du projet et perspectives futures.
- Conclusion.

I. État de l'art

Le clustering, ou segmentation non supervisée, est une technique fondamentale en *text mining* visant à regrouper des documents en ensembles homogènes sur la base de leurs similarités. Dans le contexte des catalogues de films, cette approche permet de découvrir des structures thématiques latentes, offrant une base solide pour des systèmes de recommandation plus pertinents.

- **L'algorithme k-means et ses enjeux**

A travers les méthodes de partitionnement, l'algorithme **k-means** est l'une des plus populaires en raison de sa simplicité et de son efficacité. Il opère en minimisant la variance intra-cluster (**inertie**) , chaque document étant affecté au cluster dont le **centroïde** (ou point central) est le plus proche, généralement selon la distance euclidienne. Cependant, son application au *text mining* présente des défis spécifiques. D'une part, le nombre de clusters k doit être défini a priori, ce qui nécessite des méthodes heuristiques comme la **méthode du coude** (analyse de l'inertie) ou le **score de silhouette** pour guider le choix de la valeur de K . D'autre part, l'algorithme est sensible à l'initialisation des centroïdes et tend à privilégier des clusters de forme sphérique et de taille similaire, une hypothèse qui n'est pas toujours vérifiée dans les données textuelles.

- **Vectorisation du texte : de TF-IDF aux *embeddings***

Le succès du clustering textuel repose sur la capacité à transformer le contenu sémantique en une représentation numérique exploitable. Cette étape, appelée vectorisation, est précédée d'un prétraitement rigoureux avec des techniques de *traitement automatique du langage naturel* (NLP) : tokenisation, suppression des mots-vides (*stop words*) et racinisation (*stemming* ou *lemmatization*).

Depuis longtemps , la méthode **TF-IDF (Term Frequency - Inverse Document Frequency)** a été largement adoptée pour cette tâche. Elle génère des vecteurs de grande dimension où chaque composante représente le poids d'un mot, pondéré par son importance relative dans un document par rapport à l'ensemble du corpus. Plus récemment, les approches basées sur les **plongements lexicaux (word embeddings)**, comme **Word2Vec** ou **GloVe**, ont démontré leur supériorité en capturant des relations sémantiques beaucoup plus fines. Ces modèles produisent des vecteurs denses de plus faible dimension, améliorant ainsi la cohérence des

clusters. Par rapport à notre projet, qui repose sur des caractéristiques textuelles (synopsis, genres, mots-clés) et non-textuelles (budget), la création d'une représentation vectorielle unifiée est une étape cruciale.

- **Applications dans les systèmes de recommandation de films**

Le clustering est une brique essentielle dans beaucoup de systèmes de recommandation modernes. En segmentant les films en groupes thématiques, il simplifie la tâche de suggestion en la ramenant à une recommandation intra-cluster. Le jeu de données TMDB 5000, utilisé dans ce projet, est une référence pour évaluer ces approches. Des travaux comme ceux de [ROY, PANIGRAHI, SAHA. \(2018\)](#) et [MAHESH ET KUMAR \(2022\)](#) comparent l'efficacité de différents algorithmes (**k-means**, **clustering hiérarchique**) sur ce dataset, en mesurant leur impact sur la pertinence des recommandations. D'autres études, comme celle de [WALLACE, KUPKE, KHIMJI. \(2020\)](#), explorent des approches multimodales en combinant les données textuelles avec des informations relationnelles (graphes d'acteurs), renforçant ainsi la qualité des segments découverts.

- **Limites et perspectives**

Malgré son aspect, le k-means reste limité par ses hypothèses géométriques et sa sensibilité au bruit inhérent aux données textuelles. La qualité des clusters finaux est fortement conditionnée par le choix de la valeur de **k** et la performance de la méthode de vectorisation. Pour dépasser ces limites, la recherche actuelle s'oriente vers des approches hybrides. L'utilisation de techniques de **réduction de dimensionnalité** (comme ACP ou t-SNE) avant le clustering permet de visualiser les données et de réduire le bruit. Par ailleurs, les modèles de langage profonds comme **BERT** offrent des **embeddings** contextuels encore plus puissants, ouvrant de nouvelles perspectives pour une segmentation sémantique fine. Enfin, l'intégration de **graphes de connaissances** pour modéliser les relations complexes entre films, genres et acteurs constitue une tendance prometteuse pour enrichir les modèles de clustering traditionnels.

II. Présentation, Exploration et Prétraitement des données

(Daouda Fickou)

A. Description détaillée du dataset TMDB (films, synopsis, genres, données numériques)

Le dataset TMDB contient des informations sur environ 5000 films issus de The Movie Database, une base collaborative et riche en métadonnées cinématographiques. Ce jeu de données contient 4803 lignes et 20 colonnes et est structuré autour de plusieurs types de variables :

Colonne	Type	Description
budget	Numérique (int)	Budget de production du film
genres	Catégorielle (liste)	Genres cinématographiques du film (ex. Action, Drame)
homepage	Texte (URL)	Page web officielle du film
id	Identifiant unique (int)	Identifiant unique du film dans la base
keywords	Catégorielle (liste)	Mots-clés associés au film
original_language	Catégorielle (str)	Langue originale du film
original_title	Texte (str)	Titre original du film
overview	Texte (str)	Synopsis ou résumé du film
popularity	Numérique (float)	Popularité mesurée (ex. notes, fréquentation)
production_companies	Catégorielle (liste)	Sociétés de production du film
production_countries	Catégorielle (liste)	Pays de production du film

release_date	Date (str)	Date de sortie
revenue	Numérique (int)	Recettes générées par le film
runtime	Numérique (int)	Durée du film en minutes
spoken_languages	Catégorielle (liste)	Langues parlées dans le film
status	Catégorielle (str)	Statut de production (ex. Released, Post Production)
tagline	Texte (str)	Phrase d'accroche du film
title	Texte (str)	Titre du film (localisé ou traduit)
vote_average	Numérique (float)	Note moyenne attribuée par les utilisateurs
vote_count	Numérique (int)	Nombre de votes reçus

- Les données textuelles (overview, tagline, keywords) seront nettoyées et vectorisées (TF-IDF ou embeddings) pour représenter chaque film numériquement.
- Les genres et productions serviront à classifier et analyser les films par catégories.
- Les données numériques telles que budget, popularity et vote_average seront normalisées pour le clustering.
- La date permettra d'étudier l'évolution sur le temps ou de filtrer par période.

B. Analyse exploratoire des données

L'analyse exploratoire vise à comprendre la distribution, les caractéristiques et les relations principales des variables dans le dataset TMDB. Elle permet de révéler des tendances, des anomalies éventuelles, ainsi que la structure globale des données.

1. Statistiques descriptives des variables numériques

Les principales statistiques sur des variables clés montrent :

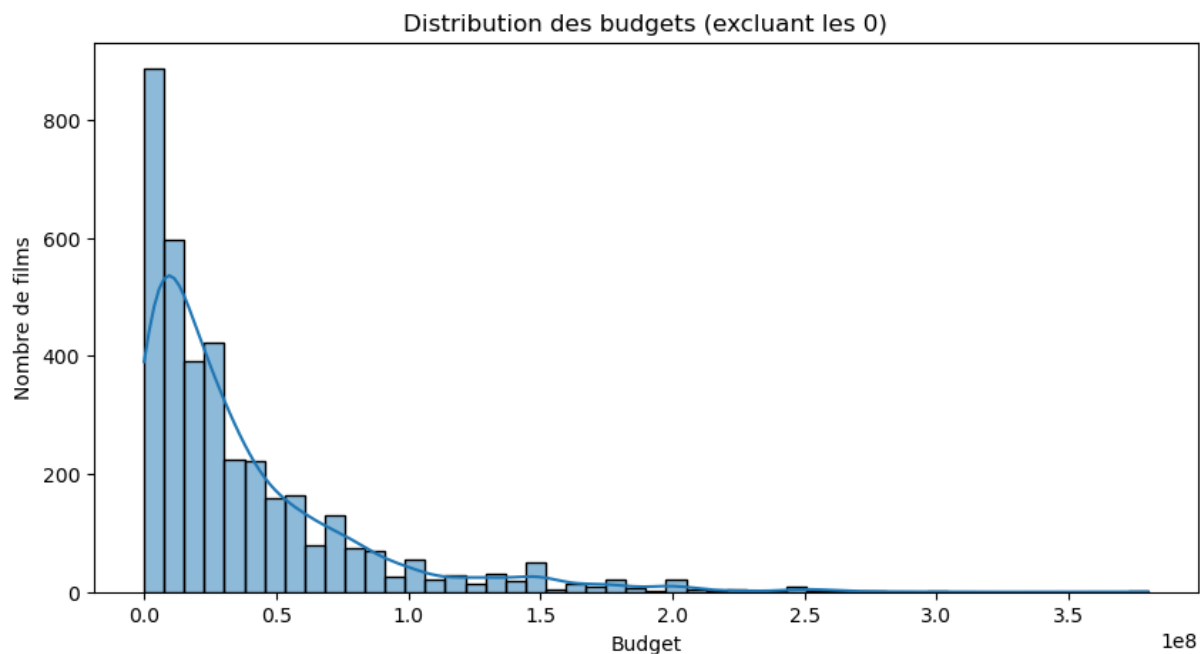
Variable	Moyenne	Écart-type	Min	Q1	Médiane	Q3	Max
budget	29 M\$	40.7 M\$	0	0.79 M\$	15 M\$	40 M\$	380 M\$
popularity	21.5	31.8	0	4.67	12.9	28.3	875.6
revenue	82.3 M\$	163 M\$	0	0	19.2 M\$	92.9 M\$	2.79 G\$
runtime	107 minutes	22.6	0	94	103	118	338
vote_average	6.1 / 10	1.19	0	5.6	6.2	6.8	10
vote_count	690	1235	0	54	235	737	13,752

Les budgets et revenus présentent des valeurs minimales à 0 et sont très dispersés, la médiane étant bien inférieure à la moyenne (effet d'outliers). La popularité varie fortement, comme le nombre de votes. Les durées sont cohérentes pour un long-métrage, mais il y a des films à 0 minute (erreur ou données manquantes).

La note moyenne des films tourne autour de 6, mais il existe quelques films extrêmes (0 ou 10).

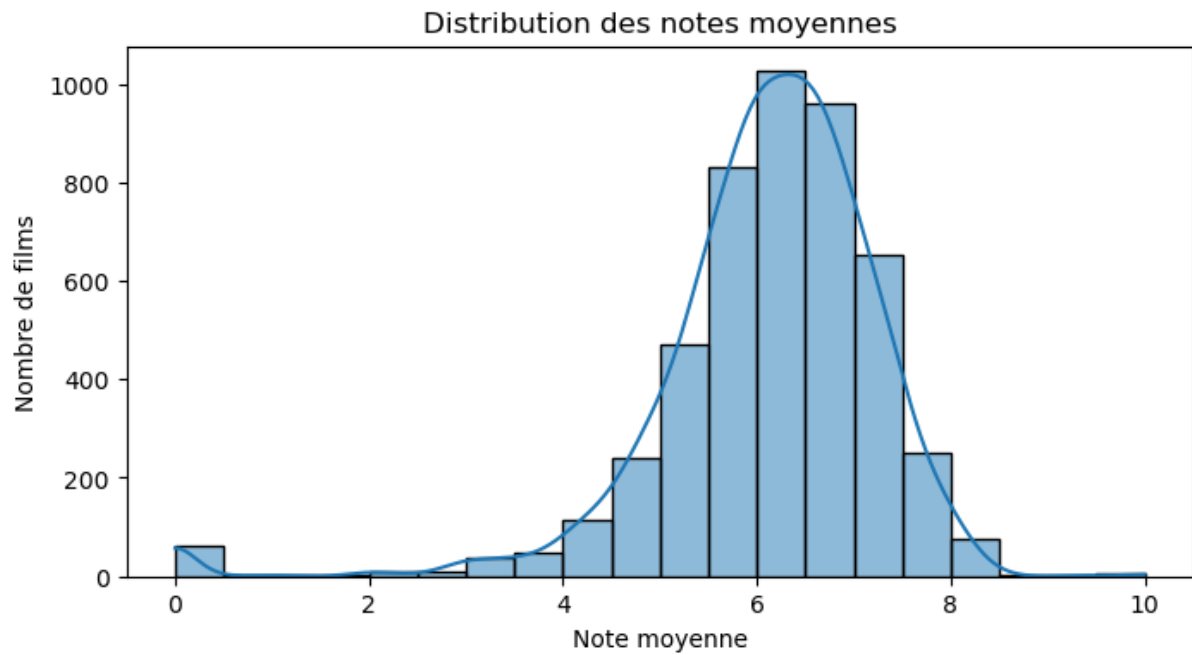
2. Visualisations principales

a. Distribution des budgets



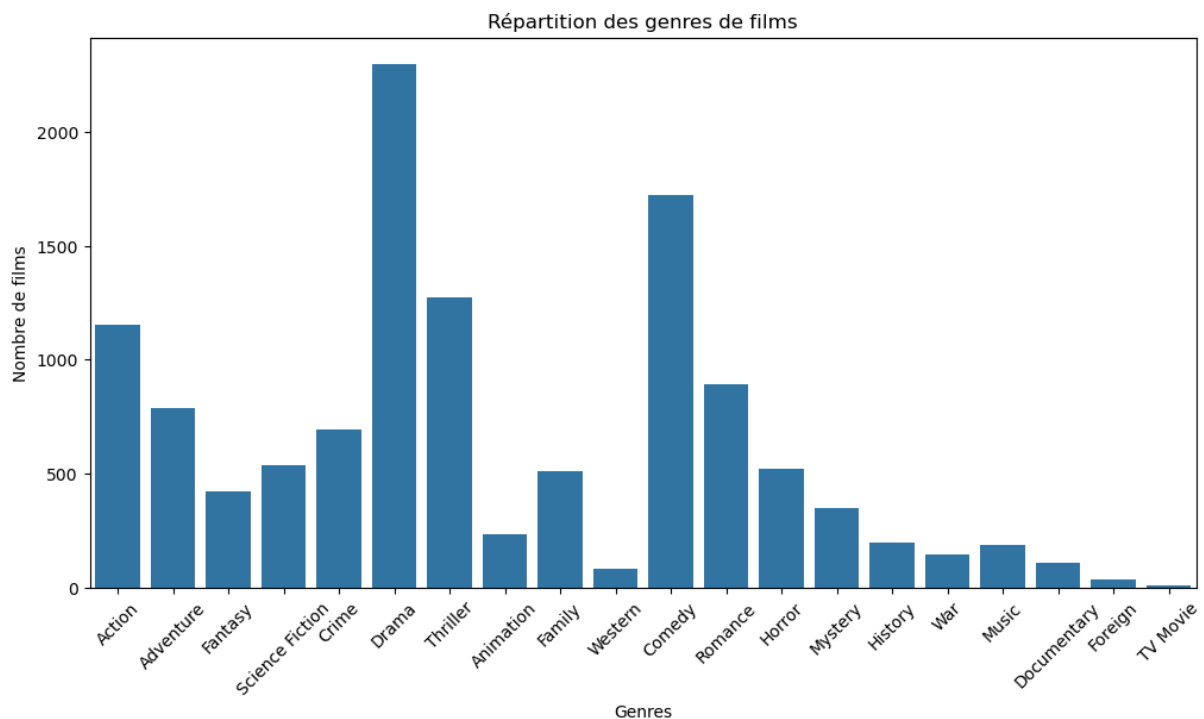
- Histogramme montrant une majorité de films à petit ou moyen budget, quelques très gros budgets rares.

b. Distribution des notes moyennes



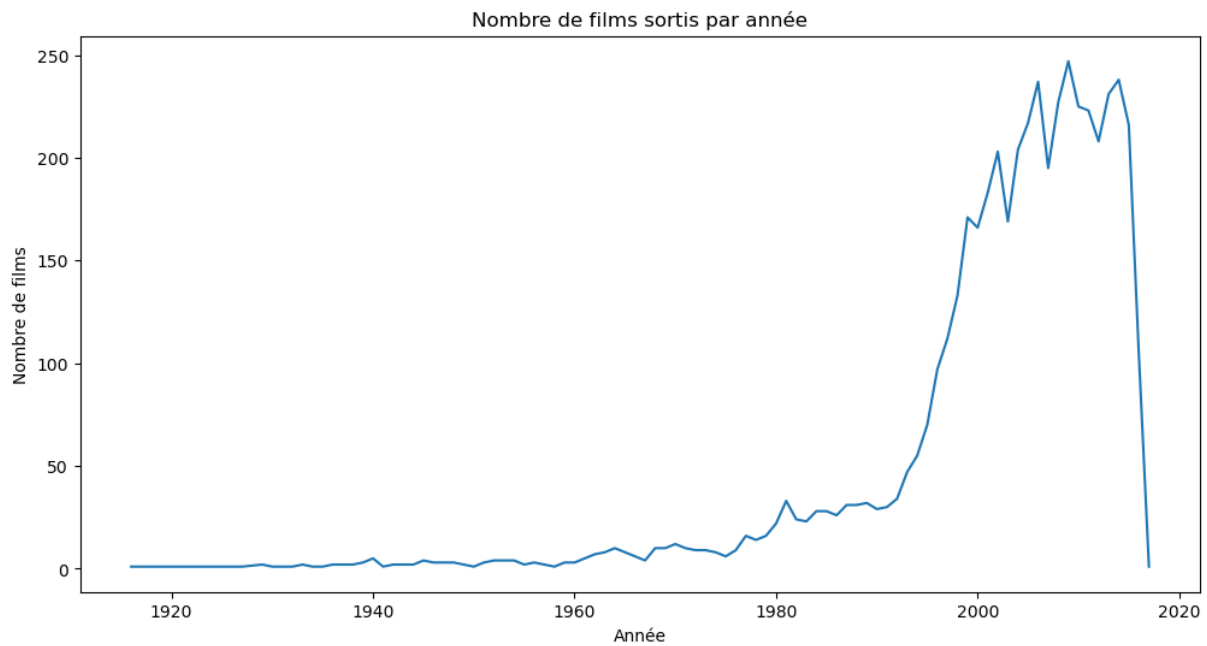
- Distribution centrée sur des notes entre 5 et 7, avec peu de films très mal ou très bien notés.

c. Répartition des genres



- Les genres les plus populaires sont Action, Drame, Thriller, Comédie.

d. Evolution du nombre de films par année



- La production de films varie selon les années, avec une tendance à l'augmentation sur les dernières décennies.

L'analyse exploratoire a permis de :

- Mettre en lumière la diversité forte des budgets, popularités et recettes.
- Confirmer la nécessité d'un traitement adapté des valeurs extrêmes (outliers).
- Identifier les genres dominants sur lesquels concentrer certaines analyses.
- Préparer le dataset pour les étapes suivantes : nettoyage approfondi, prétraitement textuel, vectorisation et clustering.

C. Nettoyage des données (valeurs manquantes, doublons)

Le nettoyage des données est une étape cruciale afin de garantir la qualité et la fiabilité des analyses ultérieures sur le dataset TMDb, qui contient des informations sur environ 5000 films. Cette étape vise à éliminer les doublons, à gérer les valeurs manquantes, et à s'assurer que les données soient cohérentes et exploitables.

1. Suppression des doublons

Un contrôle a été effectué pour détecter la présence d'entrées dupliquées.

- **Résultat** : aucun doublon n'a été trouvé, ce qui témoigne d'une bonne qualité initiale du dataset.
- **Conclusion** : aucune suppression n'a été nécessaire à ce stade.

2. Gestion des valeurs manquantes

➤ Synthèse des valeurs manquantes

Après inspection, voici les colonnes contenant des données manquantes et leur quantité respective :

Colonne	Valeurs manquantes	% par rapport au total
homepage	3,088	≈ 64%
tagline	841	≈ 17.5%
release_date	1	≈ 0.02%
Autres	0	0%

➤ Analyse

- **homepage** (URL officielle) et **tagline** (phrase d'accroche) sont des champs non critiques pour le clustering ; leur absence ne nuit pas à l'analyse principale.
- La colonne **release_date** possède une seule valeur manquante, facilement écartable si nécessaire.
- Toutes les autres colonnes essentielles (budget, genres, synopsis, scores, etc.) ne contiennent aucune valeur manquante, assurant une base solide.

➤ Traitement effectué

- Les films sans synopsis ('overview') ont été retirés (3 films), car le synopsis est fondamental pour le traitement automatique du langage et la vectorisation textuelle.
- Les valeurs à 0 dans le budget, les recettes et la durée ont été imputées par la médiane des valeurs non-nulles pour préserver l'intégrité des données numériques.

Le nettoyage a permis d'obtenir un dataset de haute qualité, prêt à être exploité pour les analyses exploratoires et les traitements avancés. L'absence de doublons et la quasi-totalité de valeurs renseignées renforcent la pertinence des résultats futurs.

Les champs non essentiels avec valeurs manquantes peuvent être soit ignorés, soit traités, sans risque de biais majeur.

D. Prétraitement textuel des synopsis

Dans le cadre de l'analyse de données issues du dataset TMDb, le prétraitement textuel des synopsis est une étape cruciale permettant d'améliorer la qualité des analyses sémantiques ultérieures, telles que la vectorisation et le clustering des films. Ce prétraitement comprend

essentiellement le nettoyage du texte, la tokenization et la suppression des mots vides (stop words).

Outils utilisés

- **NLTK (Natural Language Toolkit)** : bibliothèque Python reconnue pour le traitement du langage naturel, utilisée pour la tokenization des textes.
- **StopwordsISO** : bibliothèque fournissant des listes de stop words multilingues basées sur la norme ISO 639-1, offrant une couverture plus large que les stop words natifs de NLTK.
- **Setuptools** : utilisé pour assurer la bonne intégration de StopwordsISO dans l'environnement Python.
- **Langdetect** : bibliothèque permettant de détecter automatiquement la langue d'un texte, utilisée pour confirmer ou corriger la langue indiquée dans le dataset.

Installation des bibliothèques

Les commandes suivantes ont été exécutées pour installer les outils requis :

- `pip install nltk`
- `pip install stopwordsiso`
- `pip install setuptools`
- `pip install langdetect`

Lien vers le dépôt officiel de StopwordsISO : <https://github.com/stopwords-iso/stopwords-iso>

Description du prétraitement réalisé

1. Nettoyage du texte

- Conversion des textes en minuscules pour uniformiser la casse.
- Suppression de la ponctuation, des chiffres et des caractères spéciaux, en prenant en compte les caractères accentués via une expression régulière adaptée.

```
def nettoyage_texte(texte):  
    if not isinstance(texte, str) or pd.isna(texte):  
        return ''  
    texte = texte.lower()  
    texte = re.sub(r'^a-zââçêèëîïôûüÿñæ\s]', '', texte)  
    return texte
```

2. Détection de la langue

- Utilisation de la bibliothèque langdetect pour identifier automatiquement la langue du synopsis.
- Comparaison avec la colonne original_language fournie dans le dataset pour corriger les incohérences.
- Création d'une colonne langue_effective intégrant la langue détectée lorsqu'elle diffère de la valeur initiale.

3. Tokenization

- Découpage du texte en mots élémentaires (tokens) à l'aide de NLTK.
- Cette étape prépare le texte pour la suppression des mots non significatifs.

```
def tokenization_texte(texte):  
    return word_tokenize(texte)
```

4. Suppression des stop words

- Utilisation de **StopwordsISO** pour la suppression des mots vides, adaptée à la langue effective de chaque synopsis.
- Cette approche remplace l'utilisation des stop words de NLTK, car StopwordsISO fournit une liste plus étendue et adaptée à la diversité linguistique importante du dataset TMDB.
- La suppression ciblée des mots fréquents inutiles permet de réduire le bruit et de concentrer l'analyse sur les termes à forte charge sémantique.

```
def suppression_stopwords(tokens, langue):  
    if langue and stopwords.has_lang(langue):  
        stop_words = stopwords.stopwords(langue)  
    else:  
        stop_words = set()  
    return [mot for mot in tokens if mot not in stop_words]
```

Avantages de l'utilisation de StopwordsISO vs NLTK

- Couverture multilingue bien plus large (plus de 80 langues supportées), correspondant mieux à la diversité du dataset TMDB (37 langues identifiées).

- La détection automatique via **langdetect** permet de corriger les erreurs ou manques de métadonnées (original_language)
- Alignement sur la norme ISO 639-1 garantissant la cohérence des codes langue pour le filtrage.
- Mise à jour fréquente des listes de stop words grâce à la communauté open-source, assurant une meilleure qualité des données.

E. Vectorisation textuelle (TF-IDF ou embeddings)

Pour la vectorisation des synopsis dans le projet FilmClusterTMDB, deux approches classiques ont été étudiées :

TF-IDF

- Méthode simple basée sur la fréquence pondérée des mots dans chaque document.
- Facile à calculer et expliquer.
- Moins performant sur la capture du sens et du contexte des mots.

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
def vectoriser_tf_idf(corpus, max_termes=5000):  
    vectoriseur = TfidfVectorizer(max_features=max_termes)  
    matrice_tfidf = vectoriseur.fit_transform(corpus)  
    print(f"TF-IDF : matrice de forme {matrice_tfidf.shape}")  
    return matrice_tfidf, vectoriseur
```

Embeddings avec Word2Vec Google News 300

- Utilisation d'un modèle pré-entraîné Word2Vec avec des vecteurs de dimension 300, entraîné sur un large corpus Google News (100 milliards de mots).
- Ce modèle capture des relations sémantiques fines entre mots et expressions, plus adaptées pour le clustering et l'analyse des synopsis.

- Le modèle a été téléchargé depuis le site Huggingface : <https://huggingface.co/fse/word2vec-google-news-300/tree/main>
 - Fichier du modèle : [word2vec-google-news-300.model](#)
 - Vecteurs numpy associés : [word2vec-google-news-300.model.vectors.npy](#)

```
from gensim.models import KeyedVectors
```

```
def charger_modele_word2vec():
    print("Chargement du modèle Word2Vec pré-entraîné...")
    modele = KeyedVectors.load("../donnees/word2vec-google-news-300.model", mmap='r')
    return modele
```

```
def vectoriser_par_embeddings(corpus, modele):
    vecteurs = []
    for doc in corpus:
        tokens = tokenization_texte(doc)
        vecteurs_mots = [modele[mot] for mot in tokens if mot in modele]
        if vecteurs_mots:
            vecteur_doc = np.mean(vecteurs_mots, axis=0)
        else:
            vecteur_doc = np.zeros(modele.vector_size)
        vecteurs.append(vecteur_doc)
    matrice_embeddings = np.array(vecteurs)
    print(f"Embeddings : matrice de forme {matrice_embeddings.shape}")
    return matrice_embeddings
```

Cette méthode permet de représenter chaque synopsis par la moyenne des vecteurs mots correspondants, ce qui améliore la qualité des représentations textuelles.

Points importants

- Le modèle Word2Vec est chargé localement depuis les fichiers Huggingface, sans besoin de téléchargement à chaque exécution.
- La mémoire mappee permet de limiter la consommation mémoire lors du chargement de ce modèle volumineux (182 Mo + 3.6 Go).
- La vectorisation moyenne des mots produit un vecteur dense par document, prêt à être intégré dans le clustering, la classification.

F. Encodage et normalisation des données numériques et catégorielles

Le dataset TMDb comprend des variables de nature diverse : des variables catégorielles (genres, langue originale, pays de production) et des variables numériques (budget, popularité, revenus, durée, notes, nombre de votes). Pour préparer ces données pour un algorithme de clustering tel que k-means, il est indispensable de les transformer et mettre sur une échelle comparable.

Encodage des variables catégorielles

Les colonnes catégorielles sont encodées grâce à la fonction `encoder_categorie` qui utilise `OneHotEncoder` de `scikit-learn` :

- Les valeurs manquantes sont remplacées par la catégorie 'inconnu'.
- L'encodage one-hot est appliqué en supprimant la première catégorie par variable (`drop='first'`) pour éviter la redondance multicollinéaire.
- Le `DataFrame` initial est modifié en remplaçant les colonnes catégorielles par leurs colonnes binaires correspondantes.

Cette transformation permet de convertir des attributs qualitatifs en une représentation numérique exploitable par les algorithmes.

```
# Encodage des variables catégorielles
def encoder_categorie(df, colonnes):
    encodeur = OneHotEncoder(sparse_output=False, drop='first')
    donnees_a_encodeur = df[colonnes].fillna('inconnu')
    valeurs_encodees = encodeur.fit_transform(donnees_a_encodeur)
    noms_colonnes = encodeur.get_feature_names_out(colonnes)
    df_encode = pd.DataFrame(valeurs_encodees, columns=noms_colonnes, index=df.index)
    df_rest = df.drop(columns=colonnes)
    df_final = pd.concat([df_rest, df_encode], axis=1)
    return df_final
```

Normalisation des variables numériques

La fonction `normaliser_numerique` effectue une standardisation :

- Les valeurs manquantes sont remplacées par la moyenne de la colonne.
- `StandardScaler` transforme chaque variable pour qu'elle ait une moyenne nulle et un écart-type de 1.
- Le `DataFrame` initial est mis à jour avec ces nouvelles valeurs normalisées sur les colonnes numériques.

Cette étape permet de rendre comparables les variables d'amplitude très différente et facilite la convergence et la performance des modèles non supervisés.

```
# Normalisation des variables numériques
def normaliser_numerique(df, colonnes):
    normaliseur = StandardScaler()
    donnees_a_normaliser = df[colonnes].fillna(df[colonnes].mean())
    valeurs_normalisees = normaliseur.fit_transform(donnees_a_normaliser)
    df_normalise = pd.DataFrame(valeurs_normalisees, columns=colonnes, index=df.index)
    df_rest = df.drop(columns=colonnes)
    df_final = pd.concat([df_rest, df_normalise], axis=1)
    return df_final
```

Le prétraitement textuel des synopsis est une étape essentielle pour la réussite de l'analyse sémantique et du clustering dans le projet FilmClusterTMDB. L'utilisation conjointe de NLTK pour la tokenization et de StopwordsISO pour la suppression des stop words multilingues garantit une préparation propre et adaptée à la diversité linguistique du dataset. Cette méthodologie assure un nettoyage efficace des données textuelles, favorisant des résultats d'analyse plus précis et robustes.

III. Méthodologie et Modelisation *(Mouhamadou Gueye)*

1- Présentation de l'Algorithme Sélectionné : Clustering K-Means

Conformément aux exigences du projet qui spécifient l'utilisation d'un modèle de clustering plat, nous avons fait le choix sur l'algorithme **K-Means**. Cet algorithme est l'un des plus fondamentaux et des plus utilisés en apprentissage non supervisé. Son objectif est de partitionner un ensemble de n points de données (dans notre cas, les films représentés par des vecteurs) en K clusters distincts et non superposés. Le principe de K-Means est itératif et vise à minimiser l'inertie intra-cluster, c'est-à-dire la somme des distances au carré entre chaque point de données et le centre de son cluster, appelé **centroïde**.

Ce processus garantit que les films au sein d'un même cluster sont aussi similaires que possible, tout en étant aussi distincts que possible des films des autres clusters du model .

- **Algorithme de Lloyd** (Azencott, 2019)

Étant donnés n observations dans $\mathbb{R}^p$ et un nombre K de clusters, l'algorithme de Lloyd procède de la manière suivante :

1. Choisir K observations $\mu^{\vec{1}}, \mu^{\vec{2}}, \dots, \mu^{\vec{K}}$ parmi les n observations, pour servir de centroïdes initiaux.

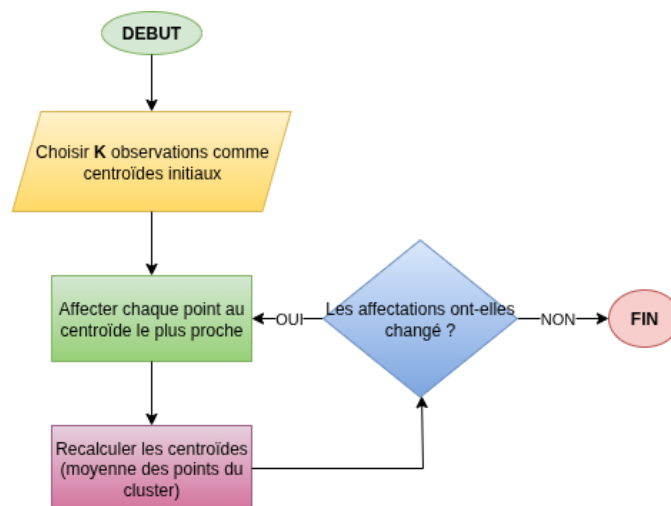
2. Affecter chaque observation $x^{\vec{i}} \in D$ au centroïde dont elle est le plus proche :

$$k(x^{\vec{i}}) = \operatorname{argmin} \|x^{\vec{i}} - \mu_k^{\vec{}}\|_2 \quad (\text{avec } k = 1, \dots, K)$$

3. Recalculer les centroïdes de chaque cluster :

$$\mu_k^{\vec{}} = (1 / |C_k|) \sum x^{\vec{i}} \quad (\text{avec } x^{\vec{i}} \in C_k)$$

4. Répéter les opérations 2-3 jusqu'à convergence, c'est-à-dire jusqu'à ce que les affectations ne changent plus.



2- Justification des Choix Méthodologiques

Pourquoi l'algorithme K-Means ?

Plusieurs raisons ont motivé le choix de K-Means pour ce projet :

- **Adéquation au besoin** : Il s'agit d'un algorithme de clustering plat, ce qui correspond parfaitement à la consigne. Il produit une partition unique et claire des données.
- **Efficacité et Scalabilité** : K-Means est réputé pour son efficacité en termes de temps de calcul, même sur des jeux de données de taille considérable. Sa complexité est quasi-linéaire avec le nombre de documents, ce qui le rend très performant.

- **Simplicité d'interprétation** : Les résultats de K-Means sont relativement simples à comprendre. Chaque document appartient à un seul groupe, et les centroïdes peuvent être interprétés comme le prototype ou le thème principal de chaque cluster .

Pourquoi les variables textuelles combinées ?

Le travail de prétraitement mené par **Fickou** a permis de créer une caractéristique textuelle unifiée (la "soupe") agrégeant le synopsis, les genres, les mots-clés, les acteurs principaux et le réalisateur. Ce choix est stratégique : plutôt que de baser notre analyse uniquement sur l'intrigue(**Overview**) , cette approche holistique permet de capturer l'ADN d'un film sur plusieurs dimensions. Un film n'est pas seulement une histoire, mais c'est aussi un style (genres), des thèmes (mots-clés) et une équipe (acteurs, réalisateur). Nous postulons que la richesse de ces informations mènera à des clusters beaucoup plus robustes et sémantiquement plus cohérents.

3- Description du Paramétrage

L'un des principaux défis de K-Means est la détermination a priori du **nombre de clusters**, c'est à dire le paramètre **K**. Un choix inadéquat de K peut mener à des regroupements peu pertinents (trop de clusters vides ou, au contraire, des clusters trop hétérogènes). Pour choisir une valeur optimale pour K, nous avons utilisé la *méthode du coude* (**Elbow Method**). Cette technique consiste à :

1. Exécuter l'algorithme K-Means pour un éventail de valeurs de K (par exemple, de 2 à 10).
2. Pour chaque valeur de K, calculer l'inertie totale du modèle.
3. Tracer la courbe de l'inertie en fonction de K.

Le "**coude**" de la courbe, c'est-à-dire le point où le gain de performance (la réduction de l'inertie) commence à diminuer de manière significative, est généralement considéré comme une **bonne estimation du nombre optimal de clusters**. Les critères d'évaluation quantitatifs, comme le *Score de Silhouette*, seront également utilisés dans la section "**Résultats**" pour valider la qualité de la partition obtenue.

4- Implémentation Technique : Outils et Librairies

Notre projet a été entièrement développé en **Python**, un langage de programmation de premier choix pour la science des données. Nous nous sommes appuyés sur un écosystème de librairies open-source robustes et reconnues :

Pandas : Pour la manipulation et le nettoyage des données tabulaires.

```
import pandas as pd
```

Scikit-learn : La librairie centrale de notre projet. Nous l'avons utilisée pour :

La vectorisation du texte avec **TfidfVectorizer**.

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

L'implémentation de l'algorithme **K-Means**

```
from sklearn.cluster import KMeans
```

```
nombre_clusters=4 # À déterminer visuellement depuis le graphique du coude
kmeans_model = KMeans(n_clusters=nombre_clusters, random_state=42, n_init=10)
# Entraînement du modèle
kmeans_model.fit(donnees_fusionnees_chargees)

if len(kmeans_model.labels_) == len(df):
    df['cluster_label'] = kmeans_model.labels_
else:
    print(f"Attention : Le nombre d'étiquettes ({len(kmeans_model.labels_)})
    ne correspond pas au nombre de lignes du DataFrame ({len(df)}).
    Impossible d'assigner les étiquettes de cluster.")

if 'cluster_label' in df.columns:
    print("Distribution des films par cluster :")
    print(df['cluster_label'].value_counts())
else:
    print("Les étiquettes de cluster n'ont pas été assignées au DataFrame.")
```

Le calcul des **métriques** et la mise en œuvre des techniques de réduction de dimension.

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.metrics import silhouette_score, silhouette_samples
```

Matplotlib et Seaborn : Pour la création de visualisations statiques, notamment le graphique

de la méthode du coude.

```
import matplotlib.pyplot as plt
import matplotlib.cm as cm
```

5- Techniques de Réduction Dimensionnelle pour la Visualisation

La matrice d'embedding issue de la vectorisation est de très grande dimension (autant de dimensions que de mots dans le vocabulaire). Il est impossible de visualiser des données dans un tel espace. Pour pouvoir représenter graphiquement nos clusters, nous avons eu recours à des techniques de réduction de dimensionnalité.

a. **ACP (Analyse en Composantes Principales - PCA)** :

Il s'agit d'une technique de transformation linéaire qui projette les données sur un nouvel espace de plus faible dimensionnalité en cherchant à maximiser la variance. Elle nous permet d'obtenir une première vue globale de la répartition des clusters.

b. t-SNE (t-Distributed Stochastic Neighbor Embedding):

C'est une technique non linéaire particulièrement efficace pour la visualisation de données de grande dimension. Contrairement à l'ACP qui préserve la structure globale, le t-SNE excelle à révéler la structure locale des données, en regroupant les points similaires et en les séparant des points dissemblables. C'est **l'outil idéal** pour vérifier visuellement la cohésion de nos clusters.

IV. Présentation, Analyse et Interprétation des resultats

(Mame Anta Bathily)

L'industrie cinématographique produit chaque année des milliers de films, dont la diversité en termes de genres, budgets et publics rend l'analyse complexe. Dans ce contexte, le clustering (ou regroupement non supervisé) permet de découvrir des structures cachées au sein des données sans recourir à une classification prédéfinie. L'objectif de ce travail est d'appliquer des techniques de clustering (K-means), combinées à des méthodes de réduction de dimension (ACP, t-SNE), sur un dataset de films provenant de TMDB. Nous cherchons à :

- Identifier des profils types de films à travers les clusters,
- Vérifier la cohérence de ces regroupements avec les genres officiels,
- Évaluer la qualité des clusters grâce à des indicateurs comme le score de silhouette,
- Enfin, explorer des pistes pour l'exploitation de ces clusters, notamment dans les domaines de la recommandation et du marketing cinématographique.

1. Interprétation qualitative des clusters

Cluster 0 – Blockbusters orientés action et super-héros

- Exemples : The Dark Knight Rises, Avengers: Age of Ultron, Man of Steel.
- Genres dominants : Drama, Action, Thriller.
- Profil : Films à budget élevé, effets spéciaux importants, grande popularité.

Cluster 1 – Films hybrides suspense/horreur

- Exemples : Suicide Squad, The Jungle Book, Shin Godzilla.

- Genres dominants : Drama, Thriller, Horror.
- Profil : Films combinant intrigue dramatique et éléments sombres, parfois horreur, destinés à un public ciblé.

Cluster 2 – Production massive et variée

- Exemples : Avatar, Pirates of the Caribbean: At World's End, Tangled.
- Genres dominants : Drama, Comedy, Action, Romance.
- Profil : Cluster le plus grand avec forte hétérogénéité, mélange de blockbusters familiaux et films populaires.

Cluster 3 – Superproductions récentes et franchises

- Exemples: Jurassic World, Furious 7, Batman v Superman, Spectre.
- Genres dominants : Drama, Action, Adventure.
- Profil : Films franchises à succès international, orientés vers le spectacle et suites cinématographiques.

2. Exploitation des caractéristiques textuelles et numériques

L'approche combinée utilisant à la fois les métadonnées et les représentations vectorielles des synopsis (ACP, t-SNE) a permis de produire des regroupements cohérents de films, reflétant des similarités thématiques et structurelles.

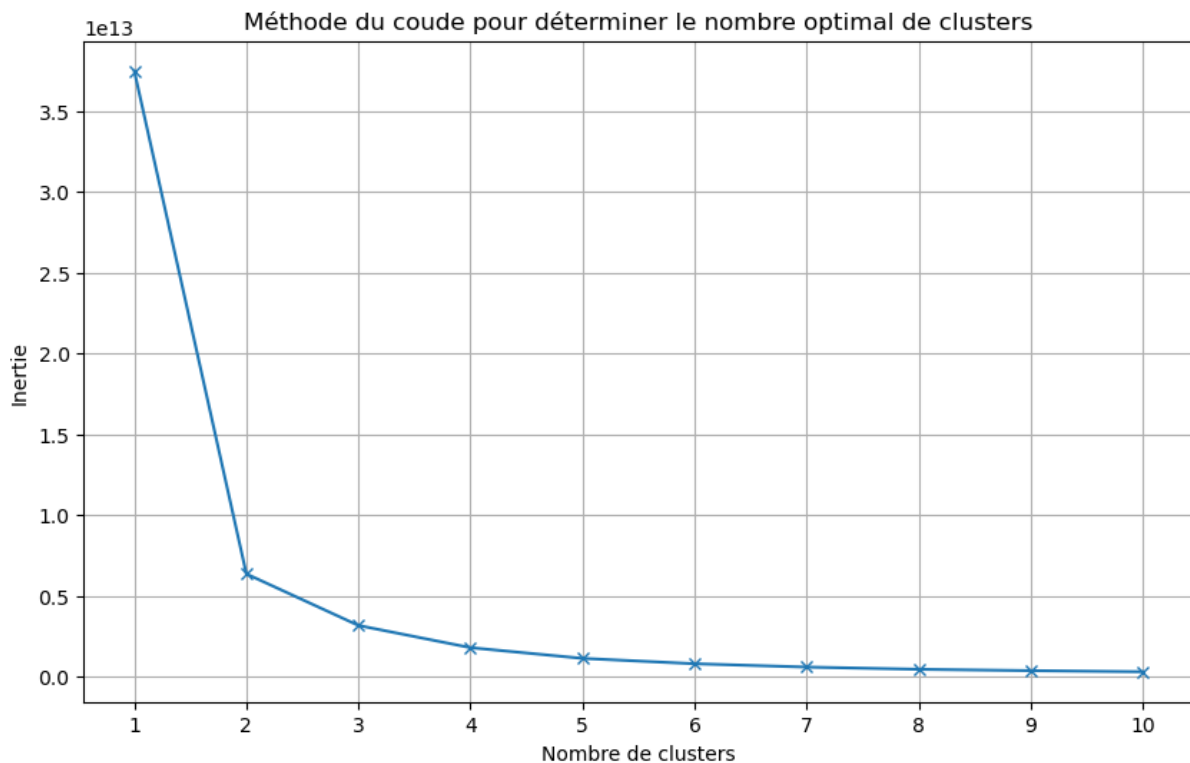


FIGURE 1 : Méthode du coude pour déterminer le nombre K optimal de clusters.

a. Détermination du nombre optimal de clusters

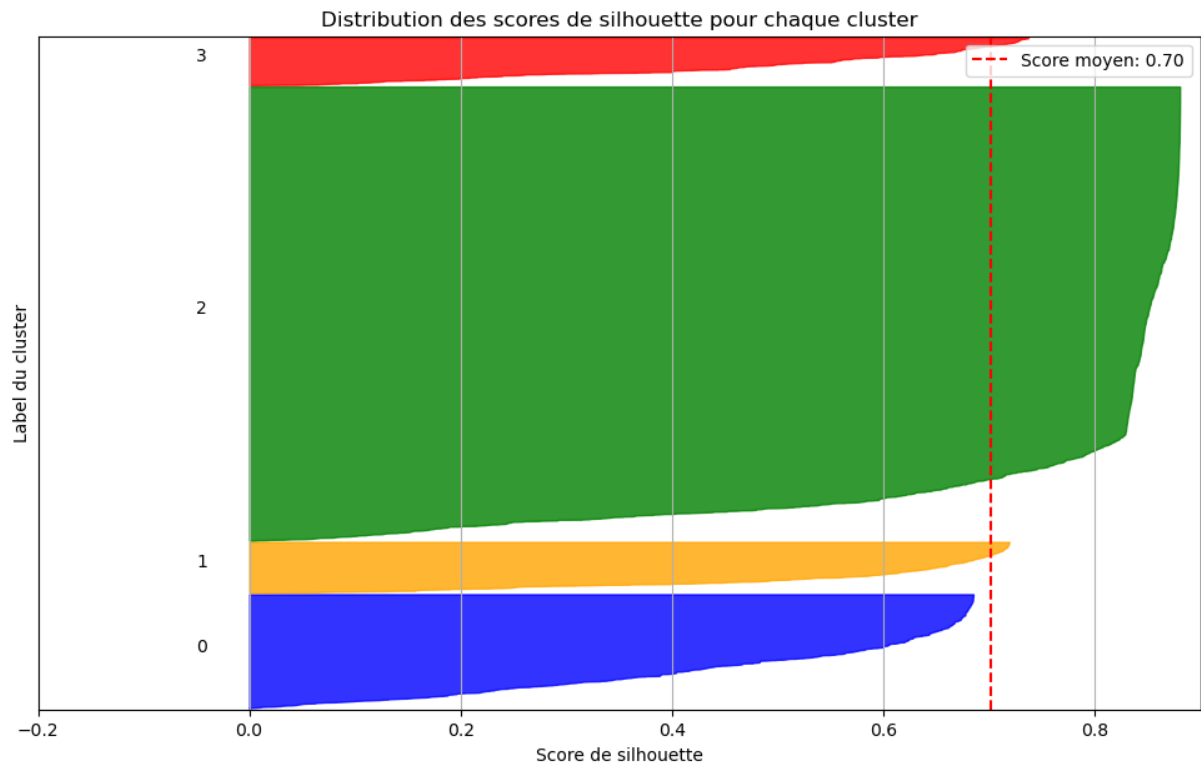
Pour choisir le nombre optimal de clusters, deux indicateurs ont été utilisés :

- La méthode du coude : elle consiste à représenter l'évolution de l'inertie intra-cluster (somme des distances entre les points et leur centroïde) en fonction du nombre de clusters K . On observe que la courbe diminue fortement jusqu'à $K = 4$, puis atteint un plateau, ce qui indique que l'ajout de nouveaux clusters n'apporte plus une amélioration significative.
- Le score de silhouette : il mesure la compacité et la séparation des clusters, avec une valeur comprise entre -1 et 1. Un score proche de 1 traduit des clusters bien séparés et homogènes, tandis qu'un score proche de 0 indique un chevauchement. Dans notre cas, un score moyen de 0,7 montre que les regroupements sont clairement définis et cohérents.

b. Correspondance avec les genres officiels

Les regroupements sont cohérents avec les genres officiels (TMDB) :

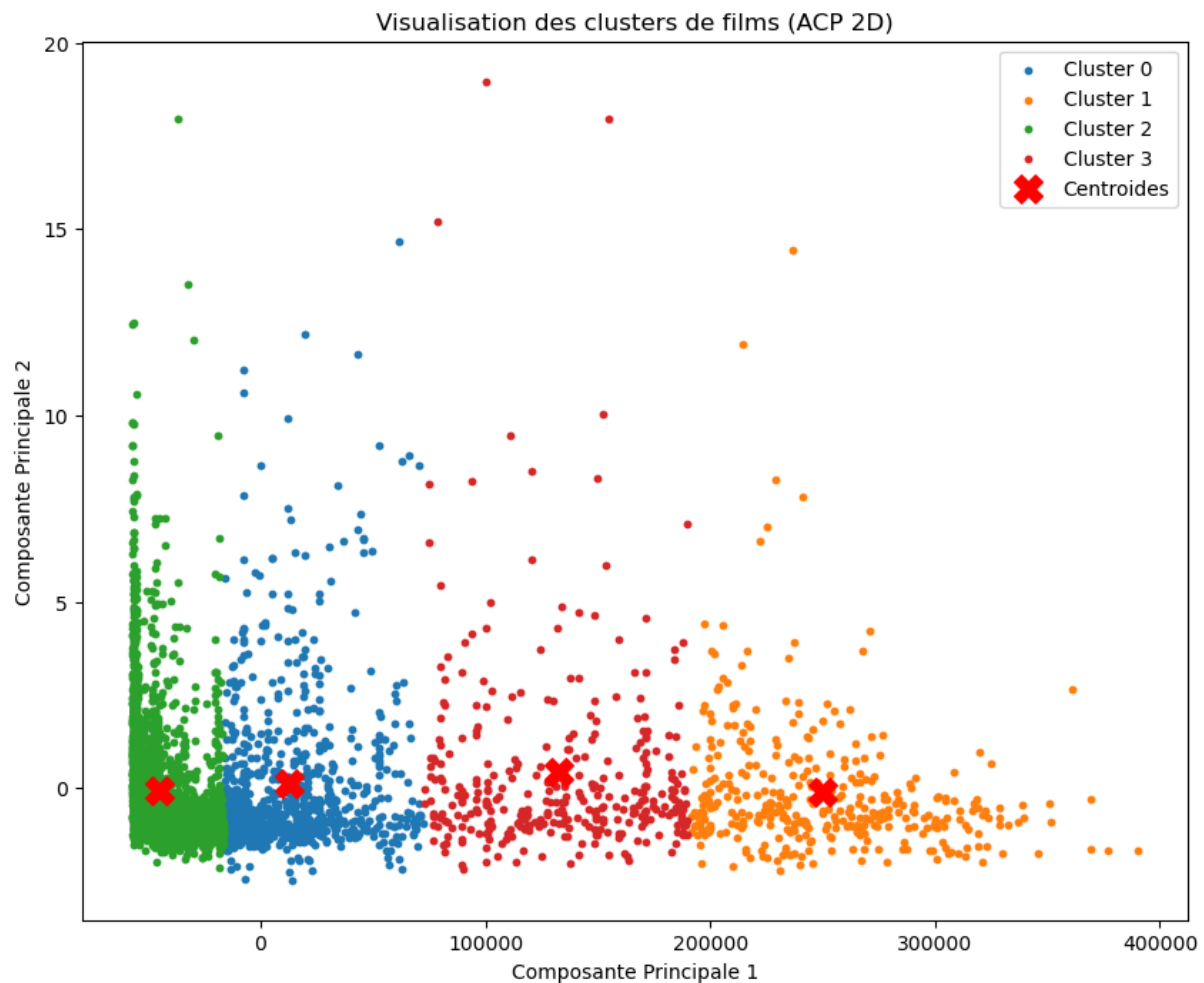
- Cluster 0 → Action / Super-héros
- Cluster 1 → Thriller / Horror
- Cluster 2 → Mix Drame – Comédie
- Cluster 3 → Action – Aventure



c. Utilisation des clusters pour recommandation ou marketing

Les clusters obtenus peuvent être exploités pour :

- Des systèmes de recommandation hybrides basés sur le contenu,
- La segmentation des publics cibles en marketing cinématographique.



3. Forces et limites de la méthode

Forces :

- K-means + ACP permet une bonne visualisation des clusters.
- Les regroupements sont pertinents et interprétables.

Limites :

- K-means suppose des clusters sphériques, ce qui ne reflète pas toujours la diversité des films.
- Le cluster 2 est très large et hétérogène, réduisant sa lisibilité.

4. Perspectives d'amélioration et pistes futures

- Tester DBSCAN ou HDBSCAN pour capter des structures plus variées.
- Utiliser des embeddings sémantiques (BERT/SBERT) pour mieux exploiter les synopses.
- Ajouter la dimension temporelle (année de sortie) pour étudier l'évolution des clusters.
- Exploiter les clusters pour améliorer la recommandation de films et le marketing ciblé.

Conclusion

L'application du clustering sur un dataset de films a permis d'identifier des profils cinématographiques distincts et cohérents avec les genres officiels. Malgré certaines limites (taille et hétérogénéité de certains clusters), ce travail montre que le clustering constitue un outil puissant pour l'analyse de données cinématographiques. Les résultats ouvrent des perspectives concrètes pour les systèmes de recommandation et la segmentation marketing dans l'industrie du cinéma.

Bibliographie

- Wallace B., Kupke C., Khimji E., "Developing Recommendation Systems for Movies Using Graph Databases," 2020.
- S. Roy, C. Panigrahi, A. Saha, "Comparative Analysis of Clustering Techniques for Movie Recommendation," MATEC Web Conf., 2018.
- Mahesh T, RVV Vinoth Kumar, "Clustering Techniques for Recommendation of Movies," Int. J. Data Informatics and Intelligent Computing, 2022.
- Lloyd, S. (1982). "Least squares quantization in PCM." IEEE Transactions on Information Theory, 28(2), 129-137.
- Article: "A novel movie recommendation method based on clustering by genre," AIP Conference Proceedings, 2019.
- Jonathan Zong, "K Means Clustering with Tf-idf Weights."
- Azencott, C.-A. (2019). *Introduction au machine learning*. 11 rue Paul Bert, 92240 Malakoff: Dunod.